

ព្រះរាជាណាចក្រកម្ពុជា

ជាតិ សាសនា ព្រះមហាក្សត្រ



INSTITUTE OF TECHNOLOGY OF CAMBODIA

Project of Compilation

Chapter 4 : Lexer generator

Group : 07

Group: I4-GIC-B

Name of Student	ID of student	Score
1. VUON SETHAR TYKUN	e20230389	…….
2. VUTHA DARALEAP	e20230820	…….
3. PHY SOPANHA	e20230530	…….
4. SOK MAKARA	e20230987	…….

Lecturer : Mr. OL Phearun (Cours and TP)

Academic year : 2026 – 2027

1. Scope

ArithLex is a lexical analyzer written with JFlex 1.9.1 and Java (JDK Temurin 25.0.4.1). It reads a UTF-8 text file of arithmetic statements such as `total=12.5+qty*2;` and prints, for each token, its type, lexeme, line and column. Positions start at 1.

Allowed characters: letters, digits, underscore, the decimal point inside a number, the symbols `= + - * / > ( ) ;` and whitespace (space, tab, carriage return, newline).

Whitespace and comments: whitespace is matched by the `WS` rule and discarded, so it produces no token, but it still counts for line and column numbers. There is no comment syntax, so comment characters such as `#` are treated as unexpected characters.

Errors and end of input: any other character is reported as an `ERROR` row with its line and column, and scanning **continues** with the next character. At the end of the input the scanner stops cleanly; an empty file prints only the header line.

Out of scope: evaluating expressions. ArithLex also does not check grammar or matching parentheses, so `3 + ;` is accepted because every token in it is valid. Negative numbers are scanned as MINUS followed by INTEGER.

2. Token definitions

The specification `arith.jflex` has 13 token types. ERROR and skipped whitespace are not counted among them.

Token	Macro or pattern	Example
INTEGER	`INTEGER = {DIGIT}+`	`12`
DECIMAL	`DECIMAL = {DIGIT}+ "." {DIGIT}+`	`12.5`
IDENTIFIER	`IDENTIFIER = {LETTER} ({LETTER} | {DIGIT})*`	`total`, `qty_2`
ASSIGN	`"="`	`=`
PLUS, MINUS, STAR, SLASH	`"+"`, `"-"`, `"*"`, `"/"`	`+ - * /`
GT, GE	`">"`, `">="`	`>`, `>=`
LPAREN, RPAREN	`"("`, `")"`	`( )`
SEMICOLON	`";"`	`;`

Helper macros: `DIGIT = [0-9]`, `LETTER = [a-zA-Z_]` and `WS = [ \t\r\n]+`. The last rule, `[^]`, matches any single character and returns ERROR.

3. Competing rules

JFlex chooses the rule that matches the longest text. If two rules match the same length, the rule written first in the file wins.

Case 1: DECIMAL vs INTEGER. For `12.5`, INTEGER matches `12` (2 characters) and DECIMAL matches `12.5` (4 characters). DECIMAL is longer, so the scanner returns one DECIMAL token. Without longest match we would get INTEGER `12`, an error for `.`, and INTEGER `5`. This is tested in B01.

Case 2: GE vs GT. For `>=`, GT matches `>` (1 character) and GE matches `>=` (2 characters). GE wins, so `a>=b` gives IDENTIFIER, GE, IDENTIFIER. A single `>` is still a GT. This is tested in B02.

Tie case. A single space is matched by both `WS` and `[^]` with the same length (1). The earlier rule, `WS`, wins, so spaces are skipped instead of being reported as errors. This is why `[^]` must be the last rule.

4. Automaton

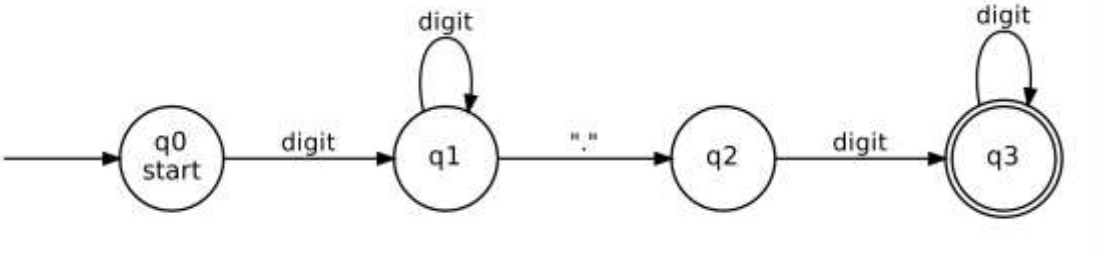


Figure 1. DFA for the DECIMAL token `{DIGIT}+ "." {DIGIT}+`. q0 is the start state and q3 is the only accepting state. Any other input leads to a dead state (not drawn).

The automaton follows the regular expression part by part. The first `{DIGIT}+` is the move from q0 to q1 on a digit and the loop on q1. The quoted dot is the move from q1 to q2. The second `{DIGIT}+` is the move from q2 to q3 and the loop on q3. Only q3 accepts, because a decimal needs at least one digit after the dot.

For `12.5` the path is q0, q1, q1, q2, q3, which ends in the accepting state, so the token is DECIMAL. For `3.` the path is q0, q1, q2 and the input ends in q2, which does not accept. The scanner goes back to the last point where a rule matched, returns INTEGER `3`, and then reports `.` as an error (test E03).

5. Testing

We recorded 15 distinct tests: 6 normal (N01-N06), 5 boundary or competing-rule (B01-B05, including the empty file, adjacent tokens and end of input right after a token) and 4 invalid (E01-E04). Each input is a text file in `tests/inputs`, with its expected output in `tests/expected`. The script `tests/run_tests.bat` runs every test and compares the scanner's output with the expected file. All 15 tests pass, and the full table is in `tests/results_table.md`.

6. Bug and its correction

Bug. While checking the output of the first working scanner, the header and the rows did not line up. The first version separated the columns with tab characters (`\t`). A tab moves to the next tab stop, so a title longer than the stop, such as `TOKEN TYPE`, pushed its column out of line with the rows below it.

Correction. We replaced the tabs with `String.format` and fixed column widths (`%-12s %-12s %-6s %s`). The format is kept in one constant, `FMT`, and the header comes from one method, `header()`, so the header and

the rows always use the same layout. After the change the columns line up, and every expected file in `tests/expected` is written in the aligned layout, so a layout regression would show up in the test comparison.

7. Contribution record

Member	ID	Role	Contribution
Vuon Sethar Tykun	e20230389	A: Scanner author	`arith.jflex` macros and rules, token table
Vutha Daraleap	e20230820	B: Driver and build	`Main.java`, build and run commands, README, coordination
Phy Sopanha	e20230530	C: Testing	Test inputs, expected outputs, results table
Sok Makara	e20230987	D: Documentation	Report, automaton drawing, slides

All members designed the token rules together and reviewed each other's work. Each member explains one rule in the presentation.